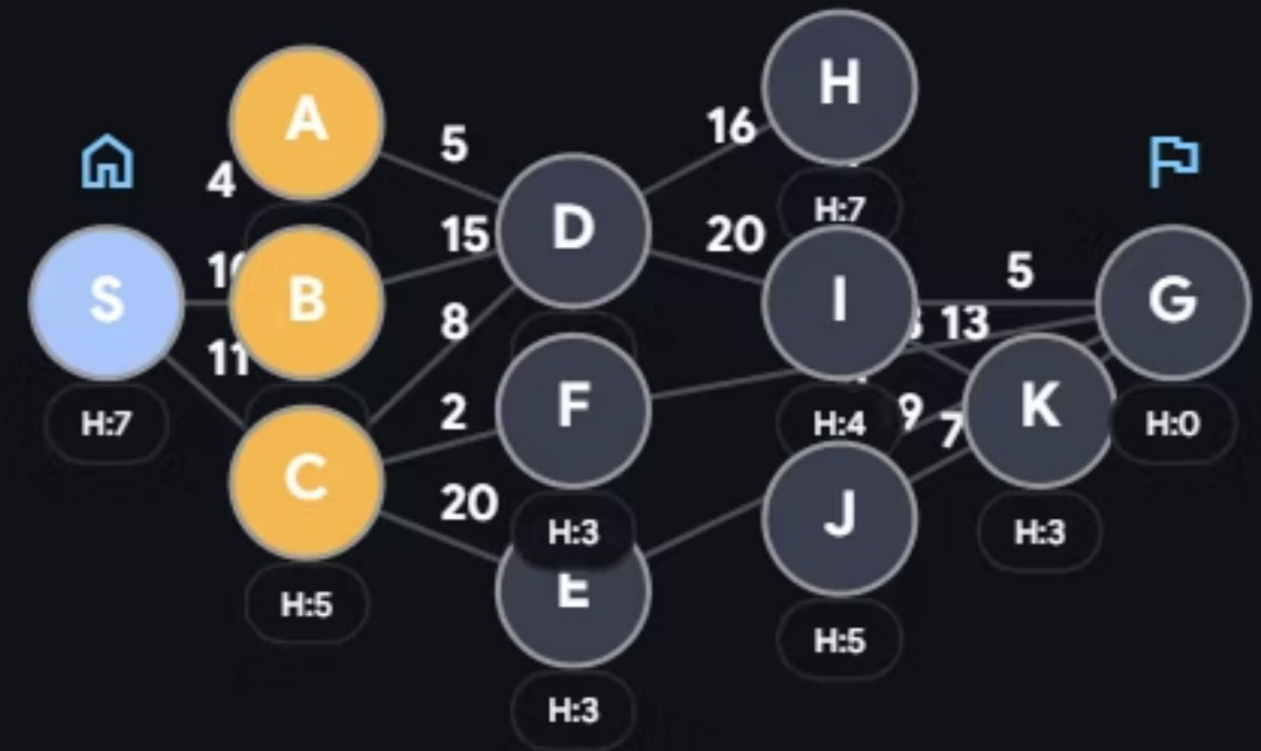


A* Pathfinding Algorithm

A* is a widely used algorithm in AI and graph theory for shortest path between 2 points. It combines the best of two worlds: the completeness of Dijkstra's algorithm and the speed of greedy best-first search, guided by a cost function incorporating a heuristic hence balancing what we know with what we estimate.



Queue (Frontier)

$f=12$ ($g=4 + h=8$)

$f=16$ ($g=10 + h=6$)

$f=16$ ($g=11 + h=5$)

Explored g-Costs

Node S: cost = 0

Node A: cost = 4

Node B: cost = 10

Node C: cost = 11

Calculations & History

Expanding S ($g=0$): A

B ($f=16$), C ($f=16$),

What Is A*?

Unlike other blind search methods, A* uses a **heuristic function** to estimate the remaining distance to the goal, allowing it to prioritise the most promising paths first (taking into account both known forward distance and potential path remaining after that)

The Core Idea

At every step, A* evaluates nodes using a combined cost function $f(n) = g(n) + h(n)$, choosing the node with the lowest total estimated cost to expand next. This is used for the priority queue

Where Is It Used?

- Video game character navigation and enemy AI
- GPS and real-time map routing systems
- Robot motion planning and obstacle avoidance
- Puzzle solvers such as sliding tile problems
- Network routing and logistics optimisation

The Cost Function: $f(n) = g(n) + h(n)$

Every node in A^* carries a score $f(n)$ i.e. the estimated total cost of the cheapest path through that node to the goal. This single value drives all decisions in the algorithm.

$g(n)$ - Cost So Far

The **actual** cost incurred from the start node to the current node n . This is a precise value and not an estimate. It grows as the path grows longer, and is updated whenever a cheaper route is discovered.

$h(n)$ - Heuristic Estimate

An **estimated** cost from node n to the goal. Common choices include Euclidean distance and Manhattan distance. The heuristic must follow one main rule - it must never overestimate the true cost.

$f(n)$ - Total Estimated Cost

The sum of both: $f(n) = g(n) + h(n)$. A^* always expands the node with the **lowest f-value** first, using a priority queue.

Step-by-Step Walkthrough

A maintains a prioritised open list of nodes to explore, based on $f(n)$. At each iteration it pops the best candidate, evaluates its neighbours, and pushes updated nodes back. The loop terminates when the goal is reached or the queue is exhausted.

01

Initialise

Create a priority queue with `(f=0, start)`.
Set `g_cost[start] = 0` and
`parent[start] = None`.

02

Pop Best Node

Remove the node with the lowest **f-value**
from the queue using `heapq.heappop(pq)`.

03

Check for Goal

If the popped node equals the goal, call
`reconstruct_path(parent, node)` and
return the path with its total cost.

04

Expand Neighbours

For each neighbour, compute `tentative_g = g_cost[node] + step_cost`.

05

Update if Better

If the neighbour is unvisited or `tentative_g` is lower than its current `g_cost`, update `g_cost`, record the parent, compute `f = tentative_g + heuristic(neighbor)`, and push to the queue.

The Algorithm in Python

Below is a clean, annotated implementation of A*. Each section of the code maps directly to the conceptual steps described earlier.

```
def astar(env, start, goal, heuristic):
    # initialize priority queue, parent and goal cost dictionary
    pq = [(0, start)]
    parent = {start: None}
    g_cost = {start: 0}
    node = None
    tentative_g = None

    # while queue is not empty, get node
    while pq:
        _, node = heapq.heappop(pq)

        # if node is goal, reconstruct
        if node == goal:
            return reconstruct_path(parent, node), g_cost[node]

        # else, get neighbours of the node, along with tentative cost
        for neighbor, step_cost in env.neighbors(node):
            # tentative cost = goal cost + step cost
            tentative_g = g_cost[node] + step_cost

            # for each neighbour, if neighbour not in visited set, or tentative cost less than goal cost
            if neighbor not in g_cost or tentative_g < g_cost[neighbor]:
                # add neighbour to goal cost dict
                g_cost[neighbor] = tentative_g
                # get total cost of neighbour (tentative goal and heuristic)
                f = tentative_g + heuristic(neighbor)
                # add current node as parent
                parent[neighbor] = node
                # add to priority queue
                heapq.heappush(pq, (f, neighbor))

    return None, float("inf")    # no path found
```

Key Data Structures

Priority Queue always surfaces the node with the lowest **f**

g_cost dict stores the best known cost to each node

parent dict enables path reconstruction after reaching the goal

Why a Priority Queue?

Using `heapq` ensures that popping the best node is **$O(\log n)$** . Without it, A* would degrade to a linear scan each iteration which is dramatically slower on large graphs.

Heuristics, Optimality & Takeaways

Choosing a Heuristic

The heuristic **$h(n)$** must be **admissible** i.e. should never overestimate the distance, to guarantee an optimal path. It should also be **consistent** where possible (ensures that the estimated cost to the goal never decreases more than the actual step cost when moving between nodes).

Manhattan - grid movement (up/down/left/right)

Euclidean - free-space, diagonal movement allowed

Chebyshev - 8-directional grid with uniform costs

A heuristic of $h(n) = 0$ reduces A* to Dijkstra's algorithm - correct but slower.

These three properties make A* the go-to choice for pathfinding - it is guaranteed to find the shortest path, will always terminate if a solution exists, and does so more efficiently than uninformed search algorithms.

Key Takeaways

A* balances **actual cost** $g(n)$ with **estimated cost** $h(n)$

A **prioritised queue** ensures the most promising nodes are explored first

An **admissible heuristic** guarantees the shortest path

The **parent dictionary** reconstructs the final path efficiently

A* is **optimal and complete** given an admissible heuristic

Optimal

Finds shortest path with admissible heuristic

Complete

Will find a solution if one exists

Efficient

Explores fewer nodes than Dijkstra using h